

Problem Set 6: Dynamic Programming II

Devin Fromond

Due: October 21st 2024

- Please type your solutions using $\text{\LaTeX}$ or any other software. Handwritten solutions will not be accepted.
- If we ask for a specific running time, a correct solution achieving it will receive full credit even if a faster solution exists. If no target running time is provided, faster solutions will earn more credit.

Problem 1: Beam Cutting (33 points)

You have finally secured permission to build a monorail across the Georgia Tech campus! To begin construction, you purchase a large linear beam whose length is the circumference of the Georgia Tech campus. However, you realize that the stations ought to use a different material than the rest of the track, so you want to slice this beam at various points to create gaps to eventually insert stations.

The beam of length n has points labeled 0 to n from left-to-right (where point 0 is at the left end and point n is at the right end). A classmate of yours does some measurements on campus to find exact locations along the beam where you should cut, and gives you a **sorted** list **cuts** of **distinct** integers at which to perform the cuts. Each cut will be on a point between 1 and $n - 1$, and any cut on one beam will split it into two beams. You can perform the cuts at any order, but must cut at exactly the locations specified (due to the color scheme, it is **not** valid to change these locations, even if the resultant component lengths are the same). The cost of cutting a component of the beam into slices of length l and r is $l + r$, but Georgia Tech makes an administrative error and accidentally gives you a budget of $l \times r$ for each such cut. This is an arbitrage opportunity!

You want to optimally order your cuts to maximize your profit, the difference between the amount of money Georgia Tech gives you and the amount of money you actually spend cutting the beam. Given n (the length of your beam), and a sorted list **cuts** of size c of integer locations at which to make the cuts, write a polynomial-time algorithm that returns the (possibly negative) maximum amount of profit you can generate.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: We define $T[i][j]$ as the maximum profit after all cuts are completed from position i to position j with regard to the beam. We define the bounds to be $0 \leq i \leq j \leq n$.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.

Solution: Base Case: If there are no cuts to be made between a segment $[i, j]$, then $T[i, j] = 0$

Recurrence Relation: We define the recurrence relation to be as follows:

$$T[i][j] = \begin{cases} 0 & \text{if } j = i + 1, \\ \max_{i < k < j} (T[i][k] + T[k][j] \\ \quad + (\text{cuts}[k] - \text{cuts}[i])(\text{cuts}[j] - \text{cuts}[k]) \\ \quad - ((\text{cuts}[k] - \text{cuts}[i]) + (\text{cuts}[j] - \text{cuts}[k]))) & \text{otherwise.} \end{cases}$$

Correctness: For every cut position k in $[i, j]$, a cut creates two segments: $[i, k]$ and $[k, j]$. After making the cut at k , the total profit is the profit of the cut added to the

maximum profits of $[i, k]$ and $[k, j]$. We consider all cut positions k and choose the k which maximizes this sum. If there are no cuts to be made, there is no associated profit nor cost.

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def beamCutting (n, cuts):
    # include endpts
    cuts = [0] + cuts + [n]
    # initialize dp table
    initialize dp of size (len(cuts))(len(cuts)) to zeros
    # iterate over whole beam
    for len in range (2, len(cuts)):
        for i in range (0, len(cuts) - 1):
            j = i + len
            T[i][j]=0
            # iterate over all possible cuts
            for k in range (i+1, j):
                l = cuts[k] - cuts[i]
                r = cuts[j] - cuts[k]
                profit = (l*r)-(l+r)+T[i][k]+T[k][j]
                if profit > T[i][j]:
                    T[i][j] = profit
    return T[0][len(c)-1]
```

- (d) Analyze the running time of your algorithm.

Solution: Very simply, we observe we have three nested *for* loops, iterating a maximum of $\text{len}(\text{cuts})$ times. We define $\text{len}(\text{cuts}) = c$. Therefore, we observe a time complexity of $O(c^3)$ since each *for* loop has a constant amount of work within them (excluding any inner *for* loops).

Problem 2: Debt Collection (33 points)

You're an upstanding citizen, and your secret stash of money has n different types of bills labeled 1 to n . For each type of bill i , it is worth v_i dollars per bill, and the number of those types of bills you have is c_i . As the upstanding citizen you are, you're trying to pay back a debt of value of exactly D dollars. You don't want to overpay this debt or underpay this debt. Design an algorithm to determine the number of different selections of all the bills to exactly pay off your debt. Note that different bills of the same type are considered identical, so two selections of bills are considered the same if and only if they use the same number of bills for every bill type.

Example: Let's say you have $n = 2$ types of bills and you need to pay a debt of $D = 15$. Bill type 1 has a value of 5 dollars per bill and you have 3 bills of this type ($v_1 = 5, c_1 = 3$). Bill type 2 has a value of 1 dollar per bill and you have 10 bills of this type ($v_2 = 1, c_2 = 10$). There are 3 selection of the bills to pay off this debt:

One bill of type 1, ten bills of type 2 ($1(5) + 10(1) = 15$)
Two bills of type 1, five bills of type 2 ($2(5) + 5(1) = 15$)
Three bills of type 1, zero bills of type 2 ($3(5) + 0(1) = 15$)

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: We define $T[i][j]$ as possible combinations to pay j dollars using only the first i types of bills. In terms of bounds, we have $1 \leq i \leq n$ and $0 \leq j \leq debt$.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.

Solution: Base Case: We have three base cases.

- 1) $T[0][0] = 1$ since there is only one way to make a total of 0 using 0 bills.
- 2) $T[0][j] = 0$ since with zero bills, the total is 0.
- 3) $T[i][0] = 1$ since there exactly one way to make a sum of 0 - 0 bills.

Recurrence Relation: We define the recurrence relation as follows:

$$T[i][j] = \sum_{k=0}^{\min(c_i, \lfloor \frac{j}{v_i} \rfloor)} T(i-1, j - k \cdot v_i)$$

Correctness: We consider all possible numbers of bills k of type i and ensure we do not exceed j through the summation's range. For each k , we take the total value from k bills of type i and subtract it from debt j .

- (c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def debtCollection(n, D, v, c):
    # initialize dp table
    initialize dp of size (n+1)(D+1) to zeros
    # note: we only do two base cases since the other includes 0's
    for i in range (0, n+1):
        T[i][0] = 1 # base case
    T[0][0] = 1 # base case
    for i in range (1, n+1):
        for j in range (1, D+1):
            T[i][j] = 0
            # hmmm what bill of i do I use?
            max_k = min(c[i-1], j // v[i-1])
            for k in range (0, max_k + 1):
                T[i][j] += T[i-1][j - k * v[i-1]]
    return T[n][D]
```

(d) Analyze the running time of your algorithm.

Solution: Our outer loop iterates over the range $(1, n)$ and therefore has a time complexity of $O(n)$. The middle loop iterates from $(0, D)$ and so, $O(D)$. The inner loop can, effectively, loop over D if $v_i = 1$ and c_i is sufficiently large. Therefore, we arrive at our time complexity of $O(nD^2)$. Since the work done inside the for loops (excluding any subsequent for loops) is constant, we arrive at a final time complexity of $O(nD^2)$.

Problem 3: Eating Bananas (33 points)

There is a monkey trying to maximize the amount of bananas it can eat. The monkey has M seconds to eat from N bags of bananas labeled 1 to N . The monkey knows that bag i has b_i bananas and it takes t_i seconds to finish all the bananas in the bag. However, the monkey cannot eat more than K bags of bananas, and each bag that the monkey chooses must have all of its bananas eaten. Design a dynamic programming algorithm to determine the maximum amount of bananas the monkey can eat.

Big Hint: Start by ignoring the K constraint, then try to add it in as a separate dimension.

- (a) Define the entries of your table in words. E.g. $T(i)$ or $T(i, j)$ is ...

Solution: We transcend 2D and use a 3D table.

$T[i][j][k]$ is the maximum number of bananas the monkey can eat by considering the first i bags, using at most j seconds, and choosing at most k bags.

- (b) State recurrence for entries of the table in terms of smaller subproblems. Briefly explain in words why it is correct. Remember to state your base case(s) as well.

Solution: Base Case: We define three base cases.

1) $i = 0$: Since no bags are considered, $T(0, j, k) = 0$.

2) $j = 0$: Since the monkey has no time to eat bananas, $T(i, 0, k) = 0$.

3) $k = 0$: Since the monkey cannot choose any bags, $T(i, j, 0) = 0$.

Recurrence Relation: We define the recurrence relation as follows:

$$T(i, j, k) = \begin{cases} \max \{T(i-1, j, k), T(i-1, j-t_i, k-1) + b_i\} & \text{if } j \geq t_i \text{ and } k \geq 1, \\ T(i-1, j, k) & \text{otherwise.} \end{cases}$$

Correctness: Taking the maximum in the recurrence relation ensures the optimal value given the two decisions (include/exclude i^{th} bag). Exclude: If the monkey does not eat from the i^{th} bag, then the max bananas is the same as $T(i-1, j, k)$. Include: If the monkey does eat from the i^{th} bag, it eats b_i bananas and uses t_i seconds.

(c) Write **pseudocode** for your algorithm to solve this problem.

Solution:

```
def eatBananas(N, M, K, b[], t[]):
    # 3D dp table
    initialize dp of size (N+1)(M+1)(K+1) to zeros

    # note: we do not need to set any base cases since
    # the table is initialized to zeros

    for i in range (1, N+1):
        for j in range (0, M+1):
            for k in range (0, K+1):
                T[i][j][k] = T[i-1][j][k] # exclude

                #include curr bag if possible
                if j >= t[i-1] AND k >= 1:
                    include = T[i-1][j - t[i-1]][k-1] + b[i-1]
                    if include > T[i][j][k]:
                        T[i][j][k] = include

    return T[N][M][K]
```

(d) Analyze the running time of your algorithm.

Solution: We observe an outer loop over N , a middle loop over M , and an inner loop over K . Since a constant amount of work is done in each for loop (excluding any additional for loops), we observe a final time complexity of $O(KMN)$.